CSE4102
Programming Languages

Lecture 16: Prolog and Logic Programming

Midterm #2, April 9

- In class (75 minutes)
- Covers:
  - Lectures 9-15 (Judgments + rules, Lambda calc, Church encodings, STLC)
  - Homeworks 3-4
- 2 double-sided 8.5x11” cheat sheets

Posted today on HuskyCT

- Practice Exam
  - Approximately same format as real exam
  - Practice exam covers the lectures and homeworks that the real exam does
  - Length, coverage of topics may not be representative
- Solutions to Practice Exam
- Reference Material
  - We will provide to you at exam

Format of Exam

- STLC Types -> Expressions, Expressions -> Types
- Build a derivation from a conclusion
- 2-3 more homework-style (not coding) questions
- No OCaml coding on exam (but understanding how lambda calculus evaluation and STLC typing works will be necessary)

Prolog

- Invented by Alain Colmerauer in 1972
- First Prolog interpreter was written in 1972 by Colmerauer and Philippe Roussel
- Programmation en logique (“programming in logic”)
- Many dialects, implementations, including an ISO standard

SWI Prolog

- Modern-ish implementation
- Largely compliant with the ISO, but adds a lot of SWI-specific features
- Free and open-source
- Available for most major platforms (including Android!?)
- Online version: SWISH (SWI for Sharing)
  https://swish.swi-prolog.org/

Prolog’s core is really simple

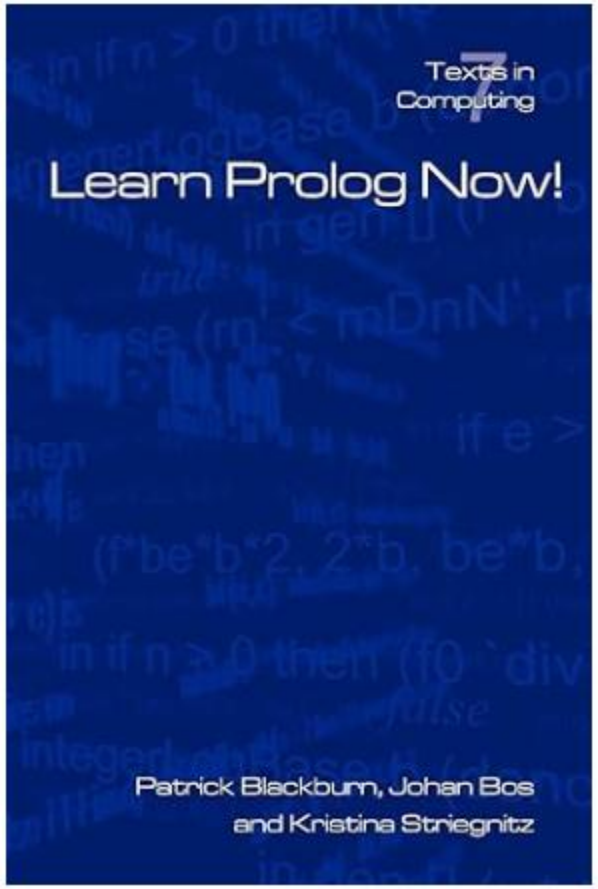
- Specify a set of **facts**
- Specify **rules** for building facts from other facts
- Pose **queries** that ask whether some fact is derivable.

To program, express facts about possible solutions

- Fact: fib(0) = 1
- Fact: fib(1) = 1
- Rule: if n > 1 and fib(n-2) = a and fib(n-1) = b, then fib(n) = a + b
- Query: What is fib(5)?

- Rule: Here's how queens attack in Chess.
- Rule: Here's what it means for a board to have *n* queens on it.
- Query: Is there an *n* x *n* board with *n* queens so no two attack each other?
- (This example is simplified quite a bit)

Textbook: Learn Prolog Now!



- Also freely available online, with companion slides.

From the LPN Chapter 1 slid

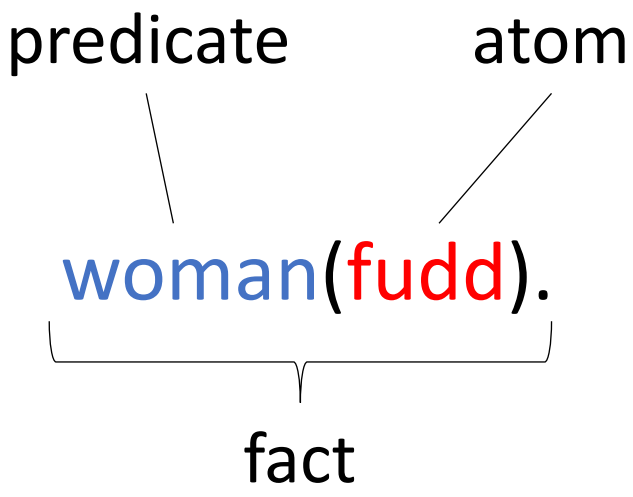
Knowledge Base 1

```
woman(mia).
woman(jody).
woman(yolanda).
playsAirGu
party.
```

Ian Bethune, the Uconn Blog

Amber Searls, Amber Searls-Imagn Images

Facts are predicates applied to atoms



Perform computation by making a query (has the same form as a fact)

```
?- woman(strong).
true.
?- man(mullins).
true.
```

Predicates have an arity (number of arguments)

- coach/1(auriemma)
- coach/2(hurley, mullins)

Prolog considers these different predicates!

Rules are like inference rules!

coach(geno, X) :- woman(X).

$$\frac{\text{woman}(X)}{\text{coach}(\text{geno}, X)}$$

Rules are like inference rules!

coach(geno, X) :- woman(X), husky(X).

$$\frac{\text{woman}(X) \quad \text{husky}(X)}{\text{coach}(\text{geno}, X)}$$

Prolog works by *unification*

woman(strong usky(strong

coach(geno, strong

?- coach(geno, strong)

Prolog grammar (simplified)

clause ::= fact. | rule. | query.

fact ::= term

rule ::= term :- term, term, …, term

query ::= ?- term

term ::= simple_term | complex_term

complex_term ::= atom(complex_term, …, complex_term)

simple_term ::= variable | atom | number

variable ::= <Uppercase identifier>

atom ::= <lowercase identifier> | ‘any string’